

I think the capital is New Delhi
Um, maybe?

Um, maybe?

Price is around 50 dollars

Review: Great battery but bad screen

Approximate value

Structured Output in LangChain

**Bridging the Gap Between GenAI
and Software Systems**

Can you calculate the root?

Perhaps...

Could be...

Not sure about that

Approximate value

Seems like

```
{  
  "city": "New Delhi",  
  "confidence": 0.98,  
  "currency": "USD",  
  "price": 50  
}
```


The Fundamental Disconnect: Human Conversation vs. Machine Logic

The Chatbot Paradigm (Unstructured)

Question: What is the capital of India?
Answer: New Delhi is the capital of India.

Free Text: Great for humans, impossible for SQL.



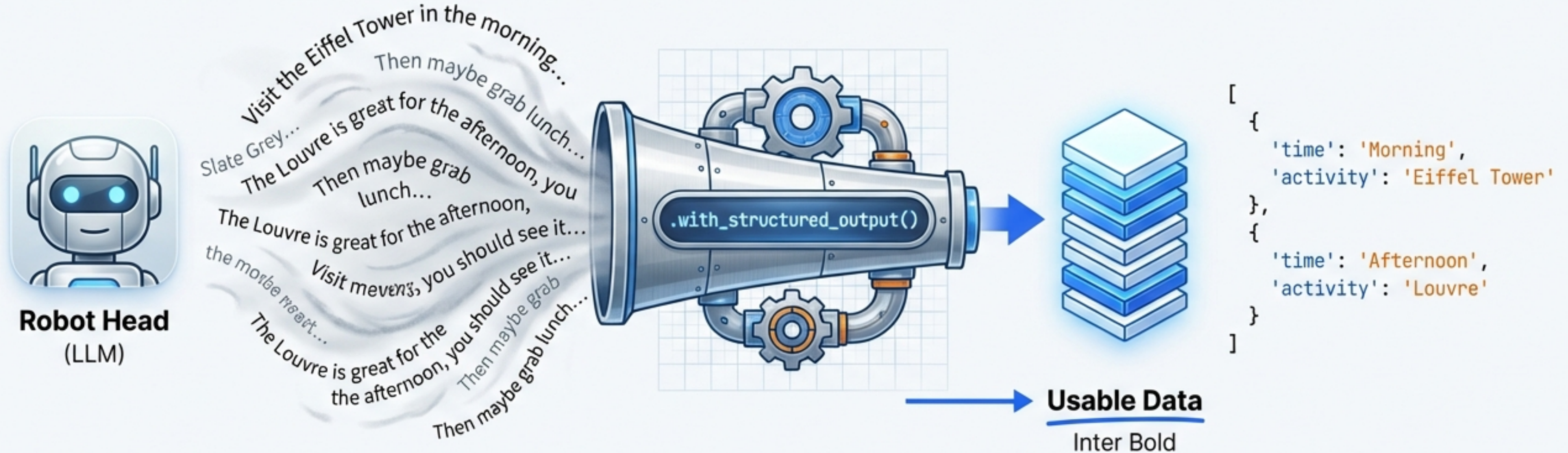
The System Requirement (Structured)

```
{  
  'city': 'New Delhi',  
  'country': 'India'  
}
```

Rigid Format: Required for Database Inserts & API Calls.

The Solution: Forcing the Model to Speak “Code”

Structured Output ensures the model returns a well-defined data format instead of free text.



Three Power Use Cases

Data Extraction



Parsing unstructured PDFs (like resumes) into database fields.

```
Input: PDF ->
Output: {'Name': 'Nitish',
        'Marks': '95%'}
```

API Development



Converting raw sentiment analysis into clean API responses for frontends.

```
Input: 'Great phone but hot' ->
Output: {'Sentiment': 'Mixed',
        'Pros': ['Great phone'],
        'Cons': ['Heating']}
```

AI Agents & Tools



Enabling Agents to use tools by formatting inputs precisely.

```
Input: 'Root of 2' ->
Output: {'func': 'sqrt',
        'val': 2}
```


The 'Magic' Behind the Scenes: System Prompt Injection

LangChain injects hidden instructions to guide the model's behaviour.

FULL PROMPT

SYSTEM PROMPT (Injected by LangChain):

You are an AI assistant that extracts structured insights...

You must return the response in JSON format matching the schema:

```
{  'summary': string,
    'sentiment': string
}.
```

`.with_structured_output()`

USER PROMPT:

Review this phone. It has great battery life...

Method 1: **TypedDict** (The Lightweight Approach)

```
from typing import TypedDict

class Person(TypedDict):
    name: str
    age: int

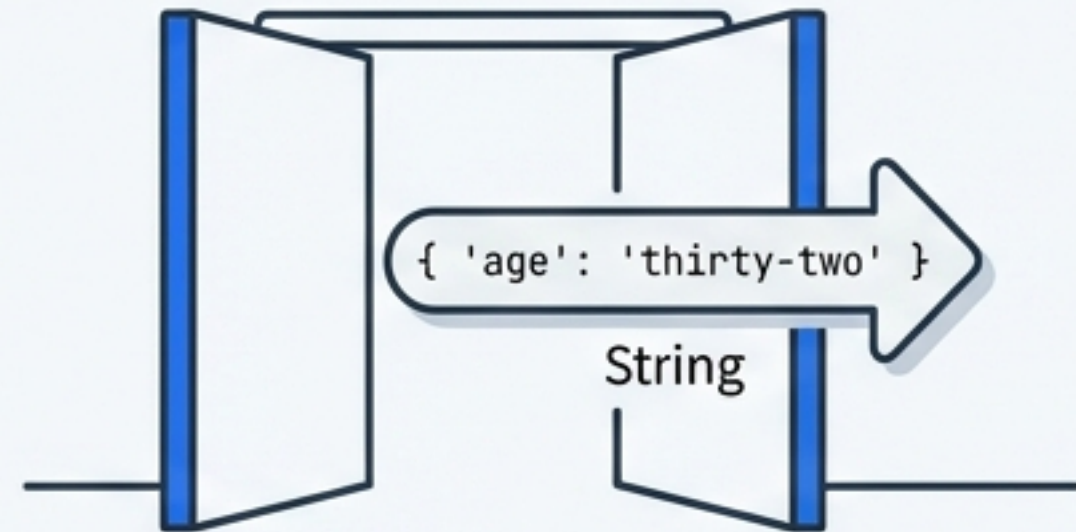
# Defining the hints for the LLM
```

Pros

- Native Python syntax.
- Great for IDE type hints.

Cons **(Critical Warning)**

- No Runtime Validation.



Python does not stop invalid types (Strings in Integer fields) from passing through.

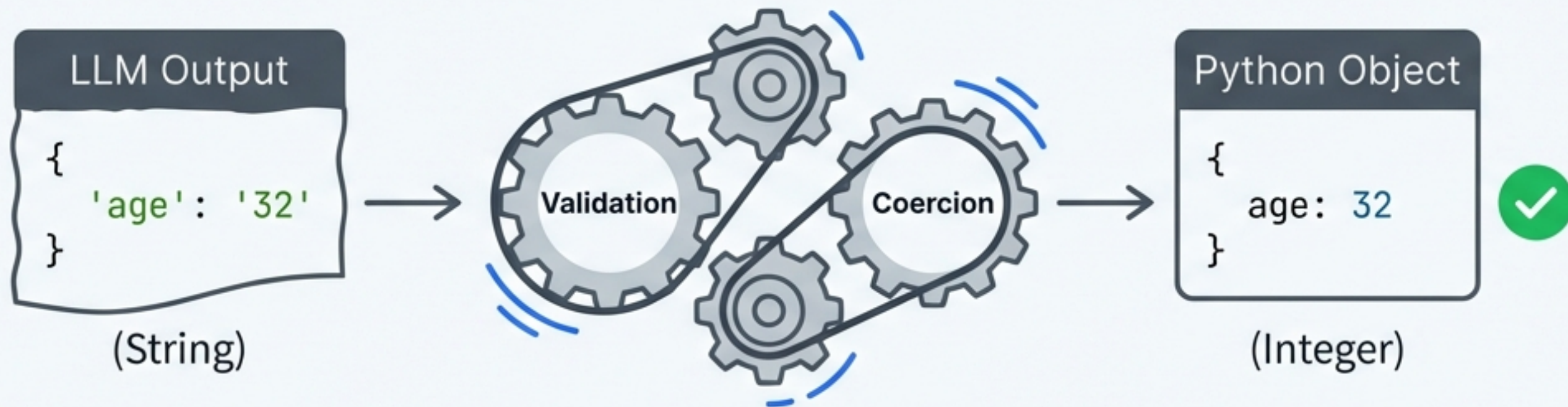
Method 2: Pydantic (The Robust Standard)

Data validation, enforcement, and type coercion.

```
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int # Pydantic enforces this type
```

The Pydantic Validator



Implicit Type Conversion: Automatically fixes compatible types.

Pydantic Deep Dive: Validation & Constraints

Fine-tuning the LLM's output with code-based prompts.

```
class Review(BaseModel):
```

```
# 1. Description acts as a prompt
```

```
summary: str = Field(description='A brief summary of the review')
```

Field Description: Passed to the LLM to explain **what** to extract.

```
# 2. Constraints enforce logic
```

```
cgpa: float = Field(gt=0, lt=10)
```

Constraints: Automatically rejects hallucinations like CGPA: 12.

```
# 3. Optional handles missing data
```

```
phone: Optional[str] = None
```

Safety Net: Prevents crashes if the LLM cannot find the data.

Method 3: JSON Schema (The Universal Language)

Necessary for cross-cross-language stacks (e.g., Python Backend + JS Frontend).

The Python Way

Concise

```
class User(BaseModel):  
    name: str
```

The Universal Way

Verbose & Explicit

```
{  
  'title': 'User',  
  'type': 'object',  
  'properties': {  
    'name': { 'title': 'Name', 'type': 'string' }  
  },  
  'required': ['name']  
}
```

Verdict: Use JSON Schema when you need language agnosticism, but stick to Pydantic for pure Python efficiency.

Decision Matrix: Which Method to Choose?

★ Best Practice

Feature	TypedDict	Pydantic (Recommended)	JSON Schema
Syntax	Simple Python	Clean Python Class	Verbose Dictionary
Validation	None (Hints only)	Robust Runtime Checks	N/A (Declarative)
Coercion	No	Yes (Auto-fix types)	No
Use Case	Quick Prototypes	Production Systems	Cross-Language Stacks

Implementation Workflow

The universal syntax: `.with_structured_output()`

1. Define Schema →

```
from pydantic import BaseModel

class Schema(BaseModel):
    name: str
    age: int
    email: str
```

2. Initialize Model →

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model='gpt-4o')
```

3. Bind Schema →

```
structured_llm = model.with_structured_output(Schema)
```

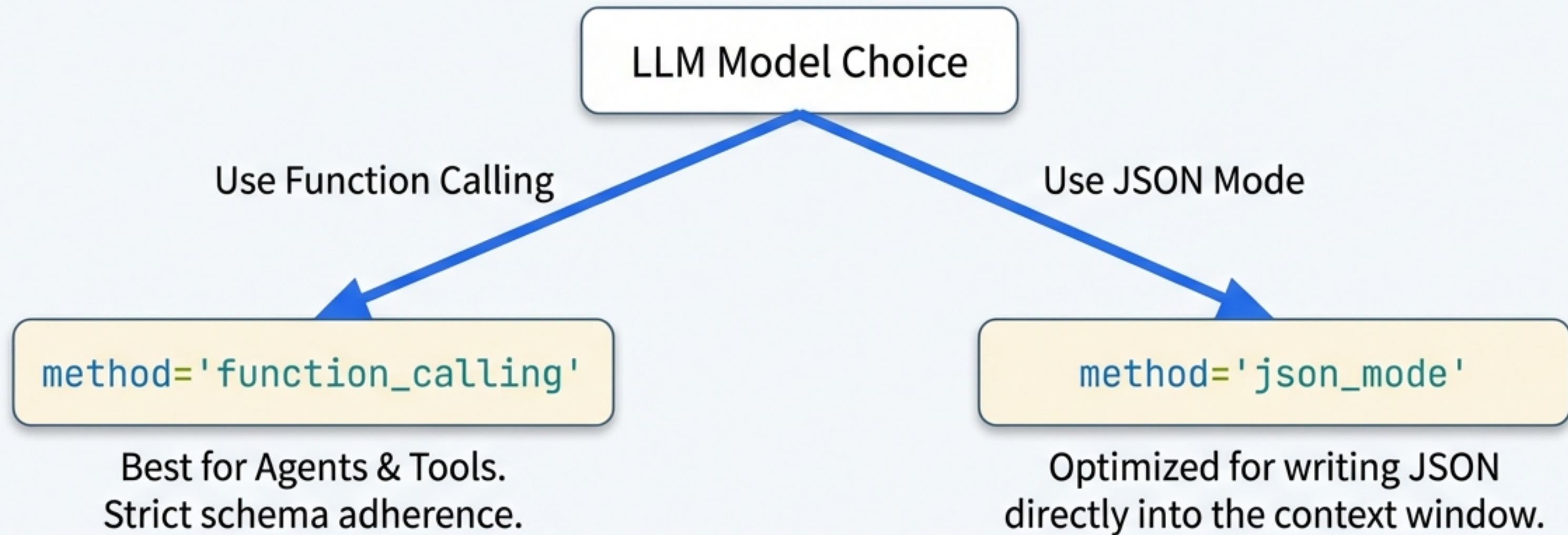
4. Invoke →

```
response = structured_llm.invoke('Review this user profile and  
extract the key information.')
```

Result: 'response' is now a validated Object, not a string.

Under the Hood: JSON Mode vs. Function Calling

Two delivery methods for structured data.



Note: Smaller open-source models (e.g., TinyLlama) may support neither. They require 'Output Parsers'.

Summary & Key Takeaways

- ✓ **The Bridge:** Structured output transforms LLMs from Chatbots into **System Components**.
- ✓ **The Tool:** Use `.with_structured_output()` for native support.
- ✓ **The Standard:** Prefer Pydantic for validation and type coercion.
- ✓ **The Fallback:** Use Output Parsers for models without native support.



Bringing It Together: The 'Review Analyzer'

```
from typing import List, Literal, Optional
from pydantic import BaseModel, Field
```

```
class Review(BaseModel):
    key_themes: List[str] = Field(description='List of key specs/topics')
```

Extracts list items like
'Snapdragon', '5000mAh'.

```
    sentiment: Literal['Positive', 'Negative']
```

Enforces exact values (Enum).

```
    pros: Optional[List[str]] = Field(default=None)
```

```
    cons: Optional[List[str]] = Field(default=None)
```

Handles missing data safely.

The Future of Integration

“We have moved beyond humans talking to machines. We are now enabling **machines to talk to other machines.”**

Next Deep Dive: **Output Parsers** (For Open Source Models).
Subscribe for the next video.